

Grafuri T3 2021

???

December 2021

1. Vom demonstra corectitudinea relatiei doar pentru S , demonstratia pentru T fiind identica.

Daca in G exista un cuplaj perfect, conform teoremei lui Hall avem:

$|N_G(A)| \geq |A|, \forall A \in S \Rightarrow |N_G(A)| - |A| \geq 0, \forall A \in S \Rightarrow \sigma(A) = 0, \forall A \in S \Rightarrow \xi(S) = 0 \Rightarrow \nu(G) = |S|$ (adevarat, deoarece G este un graf bipartit cu un cuplaj perfect).

Daca in G nu exista niciun cuplaj perfect, inseamna ca exista o multime de noduri (presupunem ca ele ar fi din S), fie ea $A^* \neq \emptyset$, a.i. $\xi(S) = \sigma(A^*) < 0$. Fie $|N_G(A^*)| - |A^*| = -k$. Adaugam in T o multime T^* de k noduri noi si conectam toate nodurile din A^* de fiecare dintre acestea. Notam cu $G' = (S, T')$ noul graf obinut. Acum, $|N_{G'}(A^*)| = |A^*|$, deci conform teoremei lui Hall in G' exista un cuplaj perfect, iar G' fiind bipartit $\nu(G') = |S| = |T'| \Rightarrow \nu(G' \setminus T^*) = |T' \setminus T^*|$. Cum $G' \setminus T^* = G$ si $|T' \setminus T^*| = |T'| - k$, avem $\nu(G) = |T'| - k$. Dar, din relatiile anterioare, $|T'| = |S|$, iar $\xi(S) = \sigma(A^*) = |N_G(A^*)| - |A^*| = -k$, deci $\nu(G) = |S| - (-\xi(S)) = |S| + \xi(S)$.

2. a) Definim $R = (G, s, t, c)$ o retea de transport astfel incat:

- $G = (\mathcal{R}, \mathcal{P}, E)$ este un digraf bipartit in care $\mathcal{R}$ este multimea cercetatorilor, $\mathcal{P}$ este multimea proiectelor, iar E este format din toate muchiile de forma $R_i P_j$, R_i are competente pentru P_j (altfel spus, $P_j \in \mathcal{P}_i$).
- Sursa s este conectata cu toate nodurile din $\mathcal{R}$, $c(sR_i) = 2$ (fiecare cercetator este alocat la maxim 2 proiecte).
- $c(R_i P_j) = 1, \forall R_i P_j \in E(G)$ ("costul" asignarii unui cercetator la un proiect este de 1).
- Toate nodurile din $\mathcal{P}$ sunt conectate de destinatia t , $c(P_j t) = n_j$ (capacitatile proiectelor).

b) O RL-alocare este obtinuta doar daca s-a gasit un flux maxim x in R a.i. toate capacitatile arcelor $P_j t$ sunt satisfacute (daca exista macar o muchie nesatisfacuta, inseamna ca acelui proiect nu i-au fost asignati destui cercetatori). Pentru a obtine repartizarea cercetatorilor este suficient sa ne uitam la valoarea fluxului pe muchiile care conecteaza $\mathcal{R}$ de $\mathcal{P}$ - daca $x_{R_i P_j} = 1$, cercetatorul R_i este asignat proiectului P_j .

c) Toate valorile fluxurilor sunt numere intregi, deci conform teoremei fluxului intreg exista un flux intreg maxim in R care poate fi gasit folosind unul dintre algoritmi urmatoari:

- Algoritmul lui Ford & Fulkerson: acesta are complexitatea de timp $O(nmU) = O((p+q) \cdot |E(G)| \cdot U)$, unde U este majorant al tuturor capacitatilor pe arce. Putem considera $U = \max(2, \max(n_j))$, dar cum $n_j < p, \forall j$ (altfel, in mod evident, nu exista nicio RL-alocare, deci nici macar nu mai este nevoie sa aplicam vreun algoritm), putem considera $U = p$. $|E(G)| = \sum |\mathcal{P}_i| \leq p \cdot q$. Deci, complexitatea algoritmului este $O((p+q) \cdot p \cdot q \cdot p = O((p+q) \cdot p^2 \cdot q)$
- Modificarea lui Edmonds & Karp: aceasta are complexitatea de $O(m^2n) = O(|E(G)|^2 \cdot (p+q)) = O((p+q) \cdot p^2 \cdot q^2)$.
- Ahuja & Orlin: modificand reseaua a.i. sa existe un preflux cu $x_{sR_i} = 2$, putem aplica algoritmul pentru a-l transforma intr-un flux maxim. Aceasta are complexitatea $O(nm + n^2 \log(U)) = O((p+q) \cdot |E(G)| + (p+q)^2 \cdot \log(U)) = O((p+q) \cdot (p \cdot q + (p+q) \cdot \log(p)))$.

Cei 3 algoritmi discutati variaza in eficienta in functie de 3 parametri: numarul de cercetatori si proiecte, numarul de competente ale cercetatorilor, numarul de cercetatori ce trebuie alocati fiecarui proiect. Se poate observa usor ca toate complexitatile sunt de forma $O((p+q) \cdot X)$, deci este suficient sa comparam $p^2 \cdot q$, $p^2 \cdot q^2$ si $p \cdot q + (p+q) \cdot \log(p)$ pentru a determina ca cel mai eficient algoritm pentru rezolvarea acestei probleme este Ahuja & Orlin.

3. a) Notam cu S, T clasele bipartiei G , a.i. $V(S) = W$ si $V(T) = W'$.

In demonstratie vom folosi urmatoarele observatii:

- S este graful complement al lui H (contine doar muchii care nu sunt in F). Evident, nodurile care formeaza o clica in H vor forma o multime stabila in S ($\omega(H) = \alpha(\overline{H}) = \alpha(S)$).
- Conform constructiei acestora, toate nodurile din S sunt conectate de toate nodurile din T , iar T nu contine nicio muchie (toate nodurile sale sunt deconectate) $\Rightarrow$ pentru a demonstra ca G este bipartit, este indeajuns sa aratam ca S nu contine nicio muchie si are macar un nod.

" $\Rightarrow$ " - Din observatiile anterioare, daca H contine o clica de cardinal k , nodurile din componenta acesteia vor forma o multime stabila in S de cardinal k . Astfel, alegand U drept restul $n - k$ nodurilor din S , in $S - U$ va fi o multime stabila, deci $G - U$ este bipartit.

" $\Leftarrow$ " - Din observatiile anterioare, daca $G - U$ este bipartit, atunci $S - U$ este o multime stabila de cardinal $n - p = k \Rightarrow$ in graful complement (H) exista o clica de cardinal k .

b) Mai intai, demonstram ca **CMP** este **NP**-hard gasind o reducere **SM** $\leq_P$ **CMP** (enuntul **SM** si demonstratia ca aceasta este **NP**-completa sunt in cursul 10, slide-urile 17-23).

Fie $G = (V, E)$ un graf si $k \in \mathbb{N}^*$ o instantă a problemei **SM**. Construim graful complement al lui G (in $O(n + m)$), apoi folosim $\overline{G}$ si k drept date de

intrare pentru problema **CMP**. Deoarece $\omega(\overline{G}) = \alpha(G)$, raspunsul la problema **SM** coincide cu cel al **CMP**.

Cum orice solutie a problemei **CMP** se poate verifica in timp polinomial (numararea nodurilor si testarea ca Q este o clica se face in $O(n + m)$ iterand prin nodurile si muchiile sale si verificand ca Q este graf complet), reiese ca **CMP** este **NP**-completa.

Pentru a demonstra ca si **NDBip** este **NP**-hard, este suficient sa gasim o reducere **CMP** $\leq_P$ **NDBip**. Aceasta reducere a fost aratata la subpunctul **a)**, unde s-a descris constructia ce trebuie facuta pornind de la o instanta de **CMP**, si cum gasirea unei submultimi U de cardinal p este echivalenta cu existenta unei clici de cardinal $k = n - p$ in graful original.

Cum orice solutie a problemei **NDBip** se poate verifica in timp polinomial (numararea nodurilor din U si testarea ca $G - U$ este bipartit se face in $O(n + m)$ incercand o 2-colorare folosind algoritmul greedy), reiese ca **NDBip** este **NP**-completa.

4. a)

```

Function getMin(List buckets)
for  $i \in 0..buckets.length$  do
  if  $buckets[i] \neq EmptyList()$  then
    return  $buckets[i][0], i$  {Toate nodurile din bucket au acelasi grad deci
    nu conteaza}
  end if
end for
end function

Function  $\delta - sort(G = (V, E))$ 
 $n \leftarrow |V|$ 
 $m \leftarrow |E|$ 
 $buckets \leftarrow EmptyList(n - 1)$  {Gradele nodurilor sunt de la 0 la  $n - 1$ }
for  $bucket \in buckets$  do
   $bucket \leftarrow EmptyList()$ 
end for
for  $v \in V$  do
   $d_G(v) \leftarrow |N_G(v)|$  { $O(1)$  daca se folosesc liste de adiacenta}
   $buckets[d_G(v)].add(v)$ 
end for
 $\pi \leftarrow EmptyList()$ 
for  $i \in 1..n$  do
   $v, d_G(v) \leftarrow getMin(buckets)$ 
   $\pi.add(v)$ 
  for  $u \in N_G(v)$  do
    {Gradele vecinilor lui  $v$  vor scadea cu 1}
     $d_G(u) \leftarrow |N_G(u)|$ 
     $buckets[d_G(u)].remove(u)$ 
     $buckets[d_G(u) - 1].add(u)$ 
  end for

```

```

    buckets[dG(v)].remove(v)
end for
π.reverse()
return π
end function

```

Algoritmul face un bucket sort, profitand de faptul ca gradele nodurilor sunt intre 0 si $n - 1$. Pentru rapiditate, se pot folosi liste (simplu/dublu) inlantuite, eventual mentinand un pointer catre ultimul element pentru a putea adauga rapid la final. Initial, popularea listei *buckets* se face in $O(n + m)$ deoarece se itereaza prin toate nodurile si se numara vecinii acestora. Apoi, popularea listei π se va face in aproximativ $O(n)$ astfel:

- Functia *getMin* returneaza nodul (si gradul sau) cu gradul cel mai mic din graful curent. Aceasta are o complexitate de $O(n)$, deoarece itereaza prin *buckets* si se opreste la primul *bucket* nevid. In practica totusi, aceasta complexitate este mai aproape de $O(1)$, deoarece nodurile vor avea numar din ce in ce mai mic de vecini, deci primele *bucket*-uri vor fi, de obicei, nevide.
- Adaugarea (la final) in lista π a nodului cu grad minim se face in $O(1)$
- Iterarea prin vecinii lui v are complexitatea $O(n)$, dar, asemanator cu *getMin*, nodurile vor avea din ce in ce mai putini vecini, deci complexitatea in practica este aproximativ constanta. Astfel se actualizeaza pozitiile acestora in *buckets*, deoarece nodul v va fi eliminat din graf.
- Toate operatiile de *add/remove* asupra *bucket*-urilor au complexitatea neglijabila deoarece, in practica, nodurile sunt distribuite uniform in *bucket*-uri si acestea au lungimi mici.
- Asemanator, eliminarea lui v se face in timp constant.

La final, se inverseaza lista π (in $O(\text{length}(\pi)) = O(n)$) si se returneaza ordonarea. Per total, complexitatea algoritmului este de $O(n + m)$.

b) Demonstram prin inductie.

Pasul de baza: Pentru $i = 1$, $\chi(G, \pi) = 1 \leq \delta_{max} + 1 = 0 + 1 = 1$

Pasul inductiv: Daca la pasul i al algoritmului de colorare greedy trebuie sa incrementam numarul de culori, inseamna ca am ajuns in situatia in care v_i este invecinat cu δ_{max_i} (δ_{max} calculat pana la pasul i) noduri colorate (in alte cuvinte, $d_{G_i}(v_i) > \delta_{max_i}$, deoarece singurele noduri colorate sunt cele care apar inaintea lui v_i in π). In acest caz, incrementam $\chi(G, \pi) \leq \delta_{max_i} + 1$.

Intuitiv, ultima incrementare a numarului de culori va fi facuta la pasul in care nodul curent are δ_{max} vecini de culori distincte, iar adaugand o culoare noua vom obtine $\delta_{max} + 1$ culori.

c) Daca G este un arbore cu macar 2 noduri, atunci cand se construiesc $\delta - ordonarea$ acestuia, se vor adauga nodurile frunza la fiecare pas (deoarece acestea au grad minim). Aceste noduri au gradul 1 (mai putin v_1 , care intotdeauna va avea gradul 0), deci $\delta_{max} = 1$. Conform **b)**, $\chi(G, \pi) \leq \delta_{max} + 1 = 2$.

Dar, cum G are macar 2 noduri, $\chi(G, \pi) > 1$. Deci, $\chi(G, \pi) = 2$. Evident, $\chi(G) = 2$, deoarece G este un arbore cu macar 2 noduri, deci $\chi(G, \pi) = \chi(G)$.

In cazul in care G are un singur nod, $\delta_{max} = 0 \Rightarrow \chi(G, \pi) = 1 = \chi(G)$ (o singura culoare pentru un singur nod).

d) Putem considera G ca fiind un arbore al carui "radacina" este formata din toate nodurile din circuit. Astfel, aplicand algoritmul de constructie al ordonarii π , mai intai se vor adauga nodurile frunza care vor avea gradul 1 (asemanator cu **c**), apoi, cand ramane doar acesta, se va adauga un nod oarecare de pe ciclu (toate nodurile au gradul 2), apoi se vor adauga pe rand toate nodurile din ciclu pornind de la capetele acestuia. Astfel, gradurile nodurilor (in subgrafurile aferente) dintr-o δ -ordonare a unui graf cu n noduri si un circuit de k sunt: $0, \underbrace{1, 1, \dots, 1}_{k-2}, 2, \underbrace{1, 1, 1, \dots, 1}_{n-k} \Rightarrow \delta_{max} = 2 \Rightarrow \chi(G, \pi) \leq 3$. Studiem urmatoarele cazuri:

- G are un singur nod: evident, $\chi(G, \pi) = \chi(G) = 1$
- G are un circuit cu numar par de noduri: conform δ -ordonarii π , mai intai se vor colora toate nodurile din circuit, unul dupa altul, in ordinea in care apar in acesta $\Rightarrow$ se vor folosi doar 2 culori pentru intreg circuitul. Apoi, se vor colora restul nodurilor urmand "crengi" din arbore, deci, asemanator **c**), se vor folosi doar aceleasi 2 culori $\Rightarrow \chi(G, \pi) = \chi(G) = 2$
- G are un circuit cu numar impar de noduri: asemanator cazului anterior, doar ca acum se vor folosi 3 culori pentru circuit (ultimul nod care trebuie colorat din circuit va avea 2 vecini de culori diferite, deci trebuie folosita o a 3-a culoare pentru acesta) $\Rightarrow \chi(G, \pi) = \chi(G) = 3$